GRADE 12 · TERM 2

Objects, Grids & Cybercrime

A pencil-and-paper workbook: design on paper first, learn the Delphi, then prove you can do it. Mid-year exam preparation.

Name: _____

Class: _____ **Date:** _____

How to use this workbook

1. Work with a **pencil**. Mistakes are how we learn: rub out and try again.
2. Start each unit with **Think First**. Try it on your own before you read the explanation.
3. Read **Learn It**, then do the activities. Trace tables and code are written **by hand**.
4. At the end of each unit, tick the "I can" boxes you are sure of. Write down anything you still want to ask.
5. The **checkpoint test** is done under test conditions, with no notes.

In this workbook

1. Blueprints and things
2. Writing a class in Delphi
3. Methods with a job
4. Using objects in a form
5. Tables of values
6. Technology and crime
7. Term checkpoint test

> **Matches the Annual Teaching Plan:** Term 2 — OOP (class, object, constructors, accessors, mutators, auxiliary methods, ToString, instantiation); 2D arrays; computer management, cloud, AI, VR/AR/MR, virtualisation; computer criminals, cybercrime, effects and safeguards.

UNIT 1 Blueprints and things

Before you write any Delphi, learn to see the world the way an object-oriented programmer does: as things, each built from a blueprint, each with something it knows and something it can do.

By the end of this unit I can: (tick each box when you are sure)

- ☐ tell a class from an object
- ☐ sort the features of a thing into attributes and methods
- ☐ explain why attributes are private and methods are public
- ☐ draw a UML class diagram using + and -
- ☐ show several objects made from one class

One thing I want to ask my teacher: _____

1.1 What it has and what it does [3]

A cellphone is a thing. Some features are **facts it holds** (an **ATTRIBUTE**, such as its colour). Others are **jobs it can do** (an **ACTION**, such as ringing). Write ATTRIBUTE or ACTION for each feature.

Feature of a cellphone	ATTRIBUTE or ACTION?
The brand name	
Make a call	
The battery level	
Send an SMS	
Charge the battery	
The list of stored contacts	

THINK FIRST · no computer needed

1.2 One plan, three tuck-shop items

The tuck shop wants a card for every item it sells. Every card has a **name**, a **price** and the number **in stock**. Fill in a card for each item: **chips**, R9,50, 24 packets; **juice**, R12, 3 bottles; **pie**, R18, 6 in the warmer.

Card	Name	Price	In stock
Card 1	_____	_____	____
Card 2	_____	_____	____
Card 3	_____	_____	____

Look at your three cards. Which parts are **exactly the same** on every card, and which parts **differ**? What would the printer call the blank card that every card is copied from?

_____

_____

_____

THINK FIRST · no computer needed

1.3 The sealed envelope game

Work in pairs. **Player A** writes three secrets on a slip of paper (name, age, favourite food), folds it and seals it in an envelope. A is now an **object** and the slip is its **private data**. **Player B** may not touch the envelope. B may only make three polite requests, and A may answer each only in the way listed:
GetAge: A reads the age aloud. **SetFood(newFood)**: A changes the food, but *only if it is a real food* (A refuses "Tuesday"). **ToString**: A reads the whole slip as one sentence.
Try it, including one silly request, then answer: (a) Why could A refuse "Tuesday"? (b) What could go wrong if B were allowed to open the envelope and write on the slip?

LEARN IT

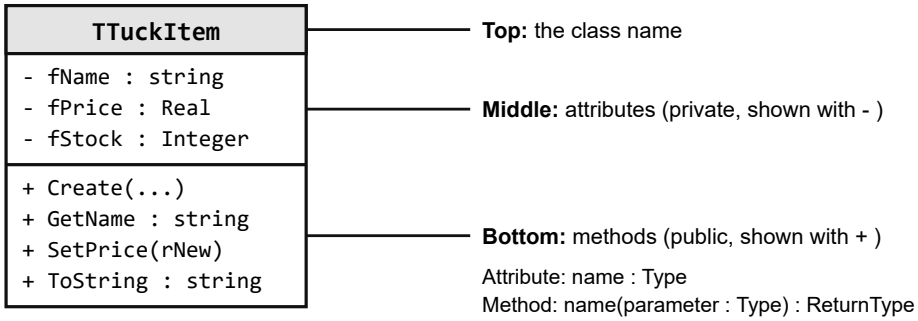
Class, object, attribute, method

A **class** is the **blueprint**: it describes what every thing of that kind knows and can do. An **object** (also called an **instance**) is one real thing built from the blueprint, with its own values. **Attributes** are the facts an object stores. **Methods** are the jobs it can do. A good class keeps its attributes **private** (hidden inside) and offers **public** methods, so the only way to read or change the data is through a method that can check the request. This idea of hiding the data behind public methods is called **encapsulation**. Delphi class names begin with **T** (TTuckItem), attribute names with **f** (fPrice) and object names with **obj** (objChips). You have used classes all year: **TButton** is a class and **btnOK** is an object.

LEARN IT

The UML class diagram

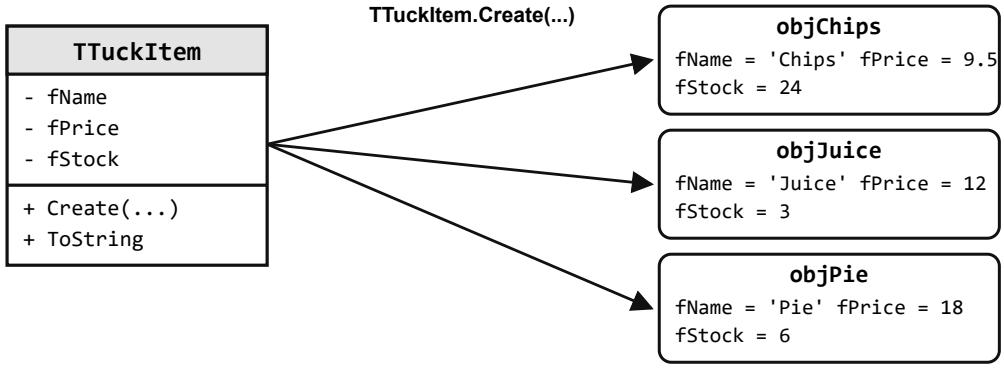
A programmer designs a class on paper first, using a **UML class diagram**: a box with three compartments. A **minus** sign means private and a **plus** sign means public.



LEARN IT

One class, many objects

The class is written **once**. Every time the program calls **Create**, a new object is built from the blueprint and given its own copy of the attributes.



1.4 Fill in the blanks [7]

A _____ is a blueprint and an _____ is one thing built from it. The facts an object stores are its _____, and the jobs it can do are its _____. In a UML diagram a _____ sign means private and a _____ sign means public. Hiding the data behind public methods is called _____. One class can be used to build _____ objects, and each has its own values.

1.5 Class or object? [6]

Circle CLASS or OBJECT.

1	TTuckItem	**CLASS**	**OBJECT**
2	objChips	**CLASS**	**OBJECT**
3	The plans an architect draws for a house	**CLASS**	**OBJECT**
4	The house at 12 Protea Road built from those plans	**CLASS**	**OBJECT**
5	TButton	**CLASS**	**OBJECT**
6	btnSave	**CLASS**	**OBJECT**

1.6 Plus or minus? [6]

A robot class has a name (text), a battery level and a step count (whole numbers). Complete the table for these members of **TRobot**.

Member of TRobot	+ or -	Attribute or method?
fBattery : Integer		
Walk(iSteps : Integer)		
fName : string		
GetBattery : Integer		
Create(sName : string)		
ToString : string		

1.7 Draw it: TRobot [6]

A **TRobot** has a name (text), a battery level (whole number) and a step count (whole number). It can be created with a name, tell its battery level, walk a number of steps, charge by a given amount, and describe itself as a sentence (ToString). Draw the UML class diagram with three compartments. Use **-** and **+**, and write the type of every attribute.

1.8 Why private? [4]

A learner writes: *fBattery := 500;* directly on a robot object from the form. Explain in two or three sentences why it is better to keep the attribute private and make the form call a method such as Charge.

UNIT 2 Writing a class in Delphi

Now the blueprint becomes code. You will read a finished class line by line, then write the pieces yourself: the declaration, two kinds of constructor, accessors and a mutator.

By the end of this unit I can: (tick each box when you are sure)

- ☐ write the type declaration of a class with private attributes and public methods
- ☐ write a parameterised and a non-parameterised constructor
- ☐ write accessor (Get) and mutator (Set) methods
- ☐ say which kind of method a header shows
- ☐ read a finished class and predict what its methods do

One thing I want to ask my teacher: _____

2.1 Interview the class [5]

Imagine you are a robot and a partner reads out the **TRobot** methods. For each, decide what the method must be **given** (its parameters) and what it **hands back** (a value, or nothing).

Method	Must be given	Hands back
GetBattery		
Walk		
Charge		
NeedsCharge		
ToString		

THINK FIRST · no computer needed

2.2 A blank book or a named book?

At the stationery shop you can buy a **blank** exercise book and write your name in it later, or order a book with your **name already printed** on the cover. Which do you choose if the shop already knows your name? Which if you are not sure whose it will be? How do these two choices compare with creating an object that already holds its values, versus creating an empty object and filling it in afterwards?

LEARN IT

The declaration: private attributes, public methods

The class goes in its own unit. The **interface** part lists everything; the **implementation** part holds the code. Attributes sit under **private**, and the methods that the outside world may call sit under **public**. Methods called **Get…** are **accessors**: they hand back a private value. Methods called **Set…** are **mutators**: they change one. **Create** is the **constructor** that builds the object: one that takes values is **parameterised**, one that takes none is **non-parameterised**. Because there are two constructors with the same name, each is marked **overload**. (Delphi writes the empty method bodies for you if you press Ctrl+Shift+C inside the declaration.)

Delphi

```
unit UTuckItem;

interface

type
  TTuckItem = class
  private
    fName  : string;
    fPrice : Real;
    fStock : Integer;
  public
    constructor Create; overload;
    constructor Create(sName: string; rPrice: Real; iStock: Integer); overload;
    function    GetName : string;
    function    GetPrice : Real;
    function    GetStock : Integer;
    procedure   SetPrice(rNew: Real);
  end;
```

2.3 Name that method [5]

Write which kind of method each header shows. Choose from: *parameterised constructor, non-parameterised constructor, accessor, mutator.*

Method header	Kind of method
constructor TTuckItem.Create;	
function TTuckItem.GetStock : Integer;	
procedure TTuckItem.SetPrice(rNew: Real);	
constructor TTuckItem.Create(sName: string; rPrice: Real; iStock: Integer);	
function TTuckItem.GetName : string;	

2.4 Fill in the blanks [8]

Attributes are declared under _____ and the methods the form may call under _____. Every attribute name starts with the letter _____. A constructor is always called _____. An accessor is a _____ because it returns a value; a mutator is a _____ because it only changes an attribute. A constructor that takes values is _____; one that takes none is _____. The private attributes can only be reached from outside through the _____ methods.

LEARN IT

The implementation

Each method body starts with the class name and a dot: **TTuckItem.GetName**. An accessor is a **function** that hands back the private value through **Result**; a mutator is a **procedure** that changes the attribute. A parameterised constructor copies its parameters into the attributes; the non-parameterised one sets defaults. **SysUtils** is needed for IntToStr and FloatToStrF. Delphi shows a decimal *comma* on South African computers (9.5 appears as 9,5), so all answers here use the comma.

Delphi

```
implementation

uses SysUtils;

constructor TTuckItem.Create;
begin
  fName  := '';
  fPrice := 0;
  fStock := 0;
end;

constructor TTuckItem.Create(sName: string; rPrice: Real; iStock: Integer);
begin
  fName  := sName;
  fPrice := rPrice;
  fStock := iStock;
end;

function TTuckItem.GetName : string;
begin
  Result := fName;
end;   // GetPrice and GetStock follow the same pattern

procedure TTuckItem.SetPrice(rNew: Real);
begin
  fPrice := rNew;
end;
end.
```

2.5 Predict: reading a finished class [4]

Use the TTuckItem class from *Learn It*. What do the two labels show once this code has run? One answer exposes a flaw in the class: which, and why?

Delphi

```
objA := TTuckItem.Create('Chips', 9.5, 24);
objB := TTuckItem.Create;
objB.SetPrice(15);
lblOne.Caption := objA.GetName + ' ' + IntToStr(objB.GetStock);
objA.SetPrice(-3);
lblTwo.Caption := FloatToStr(objA.GetPrice) + ' ' + FloatToStr(objB.GetPrice);
```

2.6 Debug: the broken coin [4]

This class has **four** errors. Find each and write the correction.

Delphi

```
type
  TCoin = class
  private
    fValue : Integer;
  public
    constructor Create(iValue: Integer);
    procedure GetValue : Integer;
    procedure SetValue(iNew: Integer);
  end;
constructor TCoin.Create(iValue: Integer);
begin
  fValue = iValue;
end;
function GetValue : Integer;
begin
  Result := fValue;
end;
procedure TCoin.SetValue(iNew: Integer);
begin
  iNew := fValue;
end;
```

2.7 Write it: four methods of TRobot [10]

Here is the declaration of **TRobot**. Write the implementation of: the **parameterised constructor** (a new robot has the given name, a full battery of 100 and 0 steps); the **non-parameterised constructor** (the robot is called **Unnamed**, with a full battery and 0 steps); the accessor **GetBattery**; and the mutator **SetName**, which must ignore an empty name.

Delphi

```
type
  TRobot = class
  private
    fName    : string;
    fBattery : Integer;       // 0 to 100 (percent)
    fSteps   : Integer;       // steps walked so far
  public
    constructor Create; overload;
    constructor Create(sName: string); overload;
    function    GetBattery : Integer;
    procedure   SetName(sNew: string);
    procedure   Walk(iSteps: Integer);
    procedure   Charge(iAmount: Integer);
    function    NeedsCharge : Boolean;
    function    ToString : string;
  end;
```

UNIT 3 Methods with a job

A good class does more than store values. Its methods make decisions, calculate, check rules and describe the object. You will write methods with a job to do, then trace calls to see exactly what happens to an object.

By the end of this unit I can: (tick each box when you are sure)

- ☐ write an auxiliary (helper) method and call it from another method
- ☐ write a method that checks a rule before it changes an attribute
- ☐ write a Boolean method
- ☐ write a ToString method
- ☐ trace calls on an object and record how its attributes change

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

3.1 The strict shopkeeper

The tuck shop keeps a card for the juice: **stock = 6**. The shopkeeper has one rule: **he sells only if he has enough**; otherwise he refuses and the card stays as it is. Customers ask for these amounts, one after the other. Write what he does and the stock on the card afterwards.

Customer asks for	Sells or refuses?	Stock after
4	_____	____
3	_____	____
2	_____	____
1	_____	____

Now write the rule as one sentence that starts with **If**.

_____

_____

LEARN IT

Auxiliary methods and calculated answers

An **auxiliary method** is a small helper that does one job for the other methods. It is normally declared **private**, because only the class itself needs it, and it usually only *works something out*: it hands back a result and changes no attribute. Here the helper builds a price as text, so **ToString** (below) need not repeat the formatting. **FloatToStrF(value, ffFixed, digits, decimals)** turns a Real into text with a fixed number of decimals. The private helper is added to the declaration under **private**: *function PriceText : string;*

Delphi

```
function TTuckItem.PriceText : string;
begin
  Result := 'R' + FloatToStrF(fPrice, ffFixed, 6, 2);   // 9.5 becomes 'R9,50'
end;
```

LEARN IT

Methods that check a rule

A mutator does not have to accept every value. It can **test first** and change the attribute only when the rule is met. **Sell** is a function that returns **True** if the sale went ahead and **False** if it did not. In a Boolean test each comparison is put in its own **brackets** when joined with **and** / **or**. Add to the declaration: *procedure Restock(iQty: Integer); function Sell(iQty: Integer) : Boolean; function IsLowStock : Boolean;*

Delphi

```
procedure TTuckItem.SetPrice(rNew: Real);
begin
  if rNew > 0 then
    fPrice := rNew;              // a price of 0 or less is ignored
end;

procedure TTuckItem.Restock(iQty: Integer);
begin
  if iQty > 0 then
    fStock := fStock + iQty;
end;

function TTuckItem.Sell(iQty: Integer) : Boolean;
begin
  if (iQty > 0) and (iQty <= fStock) then
  begin
    fStock := fStock - iQty;
    Result := True;
  end
  else
    Result := False;
end;

function TTuckItem.IsLowStock : Boolean;
begin
  Result := fStock < 5;           // a Boolean method returns the answer to a yes/no question
end;
```

LEARN IT

ToString

ToString describes the whole object in one string, ready to show in a label or memo. Join the attributes with **+**. Numbers must be changed to text first: **IntToStr** for whole numbers and **FloatToStrF** for decimals (or the helper above). For a line break between facts use **#13#10**.

Delphi

```
function TTuckItem.ToString : string;
begin
  Result := fName + ' - ' + PriceText + ' (' + IntToStr(fStock) + ' in stock)';
end;

// objChips := TTuckItem.Create('Chips', 9.5, 24);
// memOut.Lines.Add(objChips.ToString);   shows  Chips - R9,50 (24 in stock)
```

3.2 Be the computer: calls on one object [8]

The object starts as **objJuice := TTuckItem.Create('Juice', 12, 3)**, so fPrice is 12 and fStock is 3. Use the rules below. For every call write the values of **fPrice** and **fStock after** the call, and what a function call **returns** (write — if nothing is returned).

```Delphi
// Sell(q): sells only if q > 0 and q <= stock; returns True or False
// SetPrice(p): accepts p only if p > 0
// Restock(q): adds q to the stock if q > 0
// IsLowStock: True when the stock is less than 5
```

Call	fPrice	fStock	Returns
objJuice.Sell(2)			
objJuice.IsLowStock			
objJuice.Sell(2)			
objJuice.SetPrice(-5)			
objJuice.SetPrice(13.5)			
objJuice.Restock(10)			
objJuice.IsLowStock			
objJuice.ToString			

3.3 Be the computer: the robot [8]

The robot starts as **objBot := TRobot.Create('Sipho')**: battery 100, steps 0. Write the battery, the steps and any returned value after each call.

```Delphi
// Walk(n): costs 4 per step; walks only if the battery can pay for all n steps, otherwise
nothing changes
// Charge(a): battery := battery + a, but never above 100
// NeedsCharge: True when the battery is below 20
```

Call	fBattery	fSteps	Returns
objBot.Walk(10)			
objBot.Walk(12)			
objBot.NeedsCharge			
objBot.Walk(5)			
objBot.Charge(30)			
objBot.Walk(5)			
objBot.Charge(90)			
objBot.NeedsCharge			

3.4 Write it: TRobot's jobs [9]

Complete **TRobot** from Unit 2. Write the four methods from the rules above: **Walk**, **Charge**, **NeedsCharge** and **ToString**. ToString must give a sentence such as *Sipho: battery 100%, 0 steps*.

3.5 Write it: two more methods for TTuckItem [5]

(a) Write a **private auxiliary** function **StockValue** that returns the total value of the stock (price × stock) as a Real. (b) Write a public Boolean function **IsCheap** that is True when the price is **less than R10 and** there is at least one in stock.

3.6 Debug: three faulty methods [4]

These three methods of TTuckItem compile, but they do not do what they should. Find **four** errors and correct them.

Delphi

```
function TTuckItem.Sell(iQty: Integer) : Boolean;
begin
  if iQty <= fStock and iQty > 0 then
  begin
    fStock := fStock - iQty;
    Result := True;
  end;
end;

function TTuckItem.ToString : string;
begin
  Result := fName + ' costs R' + fPrice + ' and has ' + fStock + ' left';
end;

function TTuckItem.IsLowStock : Boolean;
begin
  if fStock < 5 then
    Result := 'True';
end;
```

UNIT 4 Using objects in a form

A class does nothing until a form uses it. You will declare an object variable, create the object, call its methods from a button click and show the answers in components. You will also learn the mistakes that make a program crash.

By the end of this unit I can: (tick each box when you are sure)

- ☐ declare an object variable and create the object with Create
- ☐ call methods with objectName.MethodName
- ☐ show results in labels and memos
- ☐ explain why an object that was never created crashes the program
- ☐ trace a button click that uses two objects

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

4.1 Be the objects

Three objects stand for three tuck-shop items. Their cards show: **Chips 24**, **Juice 3**, **Pie 6** in stock. **Sell(n)** sells only if there is enough stock; **Restock(n)** adds n. A refused sale changes nothing. Carry out the commands in order and write the stock on all three cards after every command.

Command	Chips	Juice	Pie
objChips.Sell(10)	____	____	____
objJuice.Sell(5)	____	____	____
objPie.Restock(4)	____	____	____
objJuice.Restock(12)	____	____	____
objChips.Sell(14)	____	____	____
objChips.Sell(1)	____	____	____

What do you notice about a command sent to one object? Does it change the cards of the other objects?

THINK FIRST · no computer needed

4.2 Put the steps in order

A programmer must use the **TTuckItem** class in a form. Here are the steps, in a jumbled order:
A objItem.Sell(3); **B** var objItem : TTuckItem; **C** objItem := TTuckItem.Create('Chips', 9.5, 24); **D** add UTuckItem to the **uses** clause **E** memOut.Lines.Add(objItem.ToString);
Write the letters in the order the programmer must follow: _____. Explain in one sentence why C must come before A.

LEARN IT

Declare, create, use

Using an object takes three steps. **1. Declare** a variable of the class type (put the unit in the **uses** clause first). **2. Create** the object with the constructor: **objChips := TTuckItem.Create(...)**. **3. Use** it by writing the object name, a dot and the method: **objChips.GetName**. The variable is only a name tag; the object is not built until Create runs, and calling a method before that crashes the program with an **Access Violation**. The constructor is called on the **class** (TTuckItem.Create), but every other method is called on the **object** (objChips.Sell).

Delphi

```
uses
  ..., UTuckItem;

var
  objChips, objJuice : TTuckItem;      // 1. declare

// inside a button click:
objChips := TTuckItem.Create('Chips', 9.5, 24);   // 2. create (parameterised)
objJuice := TTuckItem.Create;                     //    create (non-parameterised)
objJuice.SetPrice(12);                            //    then fill it in
lblName.Caption  := objChips.GetName;             // 3. use
lblStock.Caption := IntToStr(objChips.GetStock);
memOut.Lines.Add(objChips.ToString);
```

LEARN IT

A form with an object

If two buttons must use the **same** object, declare the variable once at the top of the form's unit (outside any procedure) and create the object in the button that comes first, or in **FormCreate**. If you declare it inside a procedure, the object disappears when the procedure ends. Convert text from the edit boxes with StrToInt and StrToFloat before you pass it to a method.

Delphi

```
var
  objItem : TTuckItem;                      // global: all buttons can see it

procedure TfrmShop.btnCreateClick(Sender: TObject);
begin
  objItem := TTuckItem.Create(edtName.Text, StrToFloat(edtPrice.Text),
                              StrToInt(edtStock.Text));
  memOut.Lines.Add(objItem.ToString);
end;

procedure TfrmShop.btnSellClick(Sender: TObject);
begin
  if objItem.Sell(StrToInt(edtQty.Text)) then
    lblStatus.Caption := 'Sold'
  else
    lblStatus.Caption := 'Not enough stock';
  memOut.Lines.Add(objItem.ToString);
end;
```

4.3 Will it work? [4]

Each fragment is in a button click. Circle WORKS if it runs correctly and FAILS if it crashes or will not compile.

1	objChips := TTuckItem.Create('Chips', 9.5, 24); lblName.Caption := objChips.GetName;	**WORKS** **FAILS**
2	lblName.Caption := objChips.GetName; (objChips was declared but never created)	**WORKS** **FAILS**
3	lblName.Caption := TTuckItem.GetName;	**WORKS** **FAILS**
4	objPie := TTuckItem.Create; objPie.SetPrice(18);	**WORKS** **FAILS**

4.4 Be the computer: two objects in one click [8]

Trace the code. Line numbers are in the comments. After each line write the values of the three attributes and anything that line adds to **memOut**. TTuckItem uses the rules from Unit 3; remember that a refused call changes nothing.

```Delphi
objA := TTuckItem.Create('Chips', 9.5, 24);   // 1
objB := TTuckItem.Create('Pie', 18, 5);        // 2
objA.Sell(10);                                 // 3
if objB.Sell(3) then                           // 4
  memOut.Lines.Add('Sold pies')
else
  memOut.Lines.Add('Not enough pies');
objB.SetPrice(-20);                            // 5
objB.SetPrice(20);                             // 6
if objB.IsLowStock then                        // 7
  memOut.Lines.Add('Reorder pies');
memOut.Lines.Add(objA.ToString);               // 8
memOut.Lines.Add(objB.ToString);               // 9
```

After line	objA.fStock	objB.fStock	objB.fPrice	Added to memOut
1 and 2				
3				
4				
5				
6				
7				
8				
9				

4.5 Debug: the Show button [5]

This button click should show the name of a pie, then raise its price to R20 and show whether the stock is low. Find **five** errors and write the corrected lines.

Delphi

```
procedure TfrmShop.btnShowClick(Sender: TObject);
var
  objItem : TTuckItem;
begin
  lblName.Caption := objItem.GetName;
  objItem := TTuckItem.Create('Pie', '18', 6);
  TTuckItem.SetPrice(20);
  lblPrice.Caption := objItem.GetPrice;
  if objItem.IsLowStock = 'True' then
    memOut.Lines.Add(objItem.ToString());
end;
```

4.6 Write it: create and sell [8]

The form has **edtName**, **edtPrice**, **edtStock**, **edtQty**, **lblStatus** and **memOut**. **objItem** is declared at the top of the unit. Write the code for **btnCreate** (create the object with the parameterised constructor from the three edit boxes and show its ToString in memOut) and for **btnSell** (sell the quantity in edtQty; show **Sold** or **Not enough stock** in lblStatus; show the object's ToString in memOut).

UNIT 5 Tables of values

Many things come in rows and columns: a timetable, a mark sheet, a seating plan. A two-dimensional (2D) array is the Delphi table. You will draw the grid on paper first, then learn to declare it, fill it with nested loops and add up its rows and columns.

By the end of this unit I can: (tick each box when you are sure)

- ☐ read a value from a grid using [row, column]
- ☐ declare a 2D array
- ☐ fill a 2D array with nested FOR loops
- ☐ calculate a row total, a column total and a grand total
- ☐ trace nested loops with a grid

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

5.1 The sales sheet

The tuck shop records how many items it sold each day. Work out the **total for every item** (row), the **total for every day** (column) and the **grand total**. Then check your work: the row totals added together must equal the column totals added together.

	1 Mon	2 Tue	3 Wed	4 Thu	5 Fri	Row total
1 Chips	12	15	9	14	20	____
2 Pie	8	6	10	7	18	____
3 Juice	10	12	8	11	15	____
4 Sweets	5	9	4	6	13	____
Total	____	____	____	____	____	____

What is the number in **row 2, column 5**? What does it mean? Which day was the busiest?

THINK FIRST · no computer needed

5.2 Treasure map

A grid of letters hides a word. Each clue is **[row, column]**: the **row comes first**. Find the letters for [1,2], [1,1], [3,1] and [2,1] in that order.

	1	2	3	4
1	A	K	E	T
2	I	N	D	A
3	S	O	L	U

The word is _____. Now find what [1,3] and [3,1] give. Do they give the same letter? What does this tell you about the order of the two numbers?

LEARN IT

Declaring a 2D array and reaching a cell

A 2D array has **two ranges** in the square brackets: **rows first, then columns**. Use constants for the sizes. A cell is reached with **two indices**. **array[1..4, 1..5]** has 4 × 5 = 20 cells.

arrSales	1 Mon	2 Tue	3 Wed	4 Thu	5 Fri
1 Chips	12	15	9	14	20
2 Pie	8	6	10	7	18
3 Juice	10	12	8	11	15
4 Sweets	5	9	4	6	13

Delphi

```
const
  ITEMS = 4;
  DAYS  = 5;
var
  arrSales : array[1..ITEMS, 1..DAYS] of Integer;   // row = item, column = day
  r, c     : Integer;

arrSales[2, 5] := 18;                  // row 2 (Pie), column 5 (Friday)
lblFri.Caption := IntToStr(arrSales[2, 5]);
```

LEARN IT

Filling and showing with nested loops

To visit every cell use a loop inside a loop. The **outer loop** picks the row. For each row the **inner loop** runs all the way through the columns, so every cell is visited one whole row at a time. To show the grid, build the text for one row in **sLine** in the inner loop, but add it to the memo in the outer loop, after the inner loop has finished.

Delphi

```
// fill: ask for every cell
for r := 1 to ITEMS do
  for c := 1 to DAYS do
    arrSales[r, c] := StrToInt(InputBox('Sales', 'Item ' + IntToStr(r) + ', day ' +
IntToStr(c), '0'));

// show: one memo line per row
for r := 1 to ITEMS do
begin
  sLine := '';
  for c := 1 to DAYS do
    sLine := sLine + IntToStr(arrSales[r, c]) + '   ';
  memOut.Lines.Add(sLine);
end;
```

LEARN IT

Row, column and grand totals

Ask which index stays **fixed**. A **row total** fixes the row and lets the column change (across). A **column total** fixes the column and lets the row change (down). A **grand total** needs both loops. The total must be set to 0 **before** each total you start.

Delphi

```
// row total: Juice (row 3) added across all days
iTotal := 0;
for c := 1 to DAYS do
  iTotal := iTotal + arrSales[3, c];

// column total: Friday (column 5) added down all items
iTotal := 0;
for r := 1 to ITEMS do
  iTotal := iTotal + arrSales[r, 5];

// grand total: every cell once
iTotal := 0;
for r := 1 to ITEMS do
  for c := 1 to DAYS do
    iTotal := iTotal + arrSales[r, c];
```

5.3 Fill in the blanks [9]

arrX : array[1..6, 1..3] of Integer has _____ rows, _____ columns and _____ cells. In the square brackets the _____ comes first and the _____ second. The cell in the last row and the last column is arrX[_____, _____]. To visit every cell we use _____ loops: the _____ loop handles the rows and the _____ loop the columns. For a row total the _____ stays fixed; for a column total the _____ stays fixed.

5.4 Be the computer: nested loops [6]

Trace the loop. Write the value stored in the cell on every pass, in the order the cells are visited.

Delphi

```
for r := 1 to 3 do
  for c := 1 to 3 do
    arrTab[r, c] := r * c;
```

r	c	arrTab[r, c]
1	1	
1	2	
1	3	
2	1	
2	2	
2	3	
3	1	
3	2	
3	3	

5.5 Be the computer: a row total [5]

Use **arrSales** from the grid in *Learn It*. Trace the row total of row 3.

Delphi

```
iTotal := 0;
for c := 1 to DAYS do
  iTotal := iTotal + arrSales[3, c];
```

c	arrSales[3, c]	iTotal after
1		
2		
3		
4		
5		

5.6 Predict: the forgotten reset [4]

arrG is the grid **4 2 6** (row 1) and **1 5 3** (row 2). The programmer wants the total of each row. What does memOut show? What is wrong with the code and how is it fixed?

Delphi

```
iTotal := 0;
for r := 1 to 2 do
begin
  for c := 1 to 3 do
    iTotal := iTotal + arrG[r, c];
  memOut.Lines.Add('Row ' + IntToStr(r) + ': ' + IntToStr(iTotal));
end;
```

5.7 Debug: the item totals [4]

The code should show the total of each item (row) in the form *Item 1: 70*. Find **four** errors.

Delphi

```
for r := 0 to 4 do
begin
  for c := 1 to 5 do
    iRowTotal := iRowTotal + arrSales[c, r];
  memOut.Lines.Add('Item ' + IntToStr(r) + ': ' + iRowTotal);
end;
```

5.8 **Write it: declare and fill** **[6]**

Write the constants and the declaration of the array **arrSales** (4 items, 5 days, whole numbers), the loop variables, and the nested loops that fill every cell with an InputBox.

5.9 **Write it: item totals** **[6]**

Write code that shows the total of every item in **memOut** in the form *Item 1: 70*. (The array is already filled.)

5.10 **Write it: day total, grand total and busy cells** **[9]**

Write code that: (a) shows the total sales for **Friday** (column 5) in **lblFri**; (b) shows the **grand total** in **lblAll**; (c) counts how many cells hold **15 or more** and shows the count in **lblBusy**.

UNIT 6 Technology and crime

This unit has no Delphi. It is about the machines you use every day, how to look after them, the new technologies that change how we work, and the people who attack them. Theory answers are written in **sentences**: a marker may credit only the first few points of a bullet list, but must read every word of a paragraph, so a short paragraph that gives the point, the reason and a link to the scenario can only help you.

By the end of this unit I can: (tick each box when you are sure)

- ☐ sort computer-management jobs into housekeeping, integrity and system protection
- ☐ explain cloud computing, SaaS and virtualisation in sentences
- ☐ discuss AI, including bias and accountability
- ☐ tell VR, AR and mixed reality apart
- ☐ name computer criminals and the main types of cybercrime
- ☐ explain the effects of cybercrime and the safeguards against it

One thing I want to ask my teacher: _____

6.1 Look after the machine [5]

A computer is like a house. **Housekeeping** is the tidying (clearing clutter, keeping things neat). **Integrity** is checking that nothing is cracked or corrupted. **System protection** is the locks and alarms that keep intruders out. Write H, I or S next to each job.

Job	H, I or S?
Use Disk Cleanup to delete temporary files	
Run chkdsk to find and repair errors in the file system	
Install antivirus software	
Archive last year's files on an external drive	
Watch the S.M.A.R.T. warning that a hard drive may fail	
Switch on the firewall	
Defragment a magnetic hard drive	
Plug the server into a UPS so a power cut cannot corrupt a file being saved	
Install the monthly Windows updates	

LEARN IT

Computer management

Housekeeping is the regular tidying that keeps a computer fast and roomy: **Disk Cleanup** deletes temporary files, unused programs are uninstalled, rarely used files are **archived** onto external storage or **compressed** to take less space, and a magnetic hard drive is **defragmented** so that the pieces of each file lie next to each other (never an SSD).

Integrity means that data and the file system stay correct and uncorrupted. **Chkdsk** (Disk Check) finds and repairs damaged areas, **S.M.A.R.T.** monitoring warns that a drive is about to fail, a **UPS** stops a power cut from corrupting a file that is being saved, and regular **backups** make it possible to restore a file that has been damaged.

System protection keeps unwanted people and programs out: antivirus software, a firewall, regular software updates, strong passwords and user accounts with limited rights.

6.2 Own a server, or rent one? [4]

Your school wants everyone to share documents. **Option 1:** buy a server and let the technician look after it. **Option 2:** rent storage and software over the Internet from a company. Complete the comparison.

Question	Own server	Rented (cloud)
Who buys and repairs the hardware?		
Can a learner reach files from home?		
School Internet fails: what now?		
How is the cost paid?		

LEARN IT

Cloud computing, SaaS and virtualisation

Cloud computing means renting storage, processing power or software over the Internet and paying for what you use, instead of buying and running your own equipment. **SaaS** (Software as a Service) is the part where the *software itself* is rented: you use it in a browser with nothing to install, for example Google Docs or Microsoft 365. The **advantages** are that capacity can grow or shrink when you need it, you pay only for what you use, files can be reached from any device, the provider's experts keep the software updated and backed up, and people can work on one document together. The **disadvantages** are that nothing works without an Internet connection, an outage at the provider affects every customer, and the business must trust the provider with its data.

Cloud computing has an **effect on hardware**: because the heavy work happens on the provider's servers, users can work on cheaper, low-specification computers or thin clients, and a business buys fewer servers of its own. It needs a fast, reliable connection.

Virtualisation means running several **virtual machines** on one physical computer. Each has its own operating system, memory share and software, and is separated from the others. The **benefits** are fewer physical machines to buy, power and cool, a damaged virtual machine can be restored from a saved copy, and a mistake in one machine does not harm the others.

6.3 Write it: the bakery [7]

A small bakery moves its stock records and invoices from one old office computer to a SaaS accounting program. In a short **paragraph**, discuss **two advantages** and **one disadvantage** of the move, and say what it means for the bakery's hardware.

THINK FIRST · no computer needed

6.4 Can a machine be fair?

A bank kept a record of earlier loan decisions. Look at the table, then answer below. A **computer program learns patterns from old decisions** like these and uses them to decide on new applicants.

No.	Suburb	Income	Decision
1	A	High	Approved
2	A	Middle	Approved
3	A	Middle	Approved
4	A	Low	Approved
5	A	Middle	Rejected
6	B	High	Rejected
7	B	Middle	Rejected
8	B	High	Approved
9	B	Low	Rejected
10	B	Middle	Rejected

(a) How many applicants from Suburb A were approved? _____ From Suburb B? _____

(b) Applicants 2 and 7 both have a middle income. What is different?

(c) What pattern would a program that learns from this table pick up? Would it be fair to a new applicant from Suburb B?

(d) If the program wrongly rejects an honest applicant, who should answer for it: the program, the bank, or the programmers? Give a reason.

LEARN IT

Artificial intelligence

Artificial intelligence (AI) is software that does things that normally need human intelligence, such as recognising faces, understanding speech, recommending films or spotting card fraud. **Uses** include virtual assistants, chatbots for customer service, recommendations on streaming services, fraud detection in banks, medical image analysis and self-driving vehicles.

Its **advantages** are that it works through huge amounts of data faster than a person, never gets tired, makes fewer mistakes in repetitive work and lets difficult jobs be automated. Its **limitations** are that it understands nothing, it only finds patterns in the data it was given, it needs large amounts of personal data (a privacy risk), it is expensive to build, it can replace people's jobs, and criminals can use it to make fake videos or voices.

Two limits matter most. **Bias** happens when the training data is unfair, so the AI repeats the unfairness. **Accountability** is the question of who answers when an AI decision is wrong: an AI cannot be blamed, so the company that built it or the organisation that used it is responsible, which is why important decisions should be open to human review.

6.5 VR, AR or MR? [5]

Circle **VR** (virtual reality, everything is artificial), **AR** (augmented reality, digital information laid over the real world) or **MR** (mixed reality, virtual objects that behave as if they are in the real room).

1	A trainee surgeon wears a headset and operates in a completely computer-made theatre.	**VR** **AR** **MR**
2	A phone camera shows a virtual sofa on the floor of your real lounge.	**VR** **AR** **MR**
3	Through a headset a virtual ball bounces off your real desk and rolls under your real chair.	**VR** **AR** **MR**
4	Arrows for the route are projected onto the real road in front of a driver.	**VR** **AR** **MR**
5	A player in a headset walks on a treadmill through a dragon's cave that does not exist.	**VR** **AR** **MR**

LEARN IT

Virtual, augmented and mixed reality

Virtual reality (VR) places the user inside a completely computer-made world and shuts out the real one. It needs a headset with a screen for each eye, motion sensors or trackers, controllers or data gloves, headphones, and a powerful processor and graphics card to draw the scene instantly. It is used for training (pilots, surgeons), education, gaming and treating phobias. Its **limitations** are that the software is built for one purpose, it cannot easily copy a changing real situation, and long use can lead to addiction and social isolation.

Augmented reality (AR) keeps the real world in view and adds digital information to it, for example furniture shown in your room through a phone, or navigation arrows on a windscreen. It needs a camera and sensors to see where things are, a screen or glasses, a battery that lasts, Wi-Fi or Bluetooth, and a fast processor. Its **limitations** are that drawing realistic graphics at the right size and angle is hard, and a phone has little storage, power and battery for it.

Mixed reality (MR) goes further: the virtual objects are *anchored* in the real room and react to it, so a virtual ball can bounce off a real table. It needs the sensors of AR and the processing power of VR, so the equipment is costly.

THINK FIRST · no computer needed

6.6 Spot the scam

Learners at a school received these messages. Look at each carefully and **underline or circle the warning signs**. Then list at least four signs below.

> **SMS from 071 555 0199:** YourBank: Ur acount is BLOCKED!! Click http://yourbank-verify.example/unblock within 2 hours or lose ALL your money. Reply with your PIN to confirm.

> **Email from support@yourbank-secure.example:** Dear Customer, you have won a R50 000 prize! To claim it, enter your ID number, card number and CVV on our secure page. Offer ends today.

LEARN IT

Computer criminals and the crimes they commit

A **hacker** gains access to a system without permission; *white-hat* hackers are hired to test security, *black-hat* hackers are criminals. A **cracker** breaks the protection on software or encryption in order to use or share it illegally. A **cyber gang** is an organised group who share the work of large attacks, such as a scam that needs writers of malware, people who steal data and people who sell it.

Crime	What it is
Identity theft	Stealing someone's personal details to pretend to be them, for example to open an account or take a loan in their name.
Phishing	Fake emails, SMSs or web pages that look genuine and trick the victim into giving away passwords or card details.
Ransomware	Malware that locks or encrypts the victim's files and demands payment to unlock them.
Botnet	A network of infected computers (*zombies*) secretly controlled by a criminal, often used to flood a website with traffic (a DDoS attack) or to send spam.

The **effects** on a person include money stolen, a ruined credit record, shame and stress, and having to prove they are not the criminal. A business can lose money and customers' trust, may be fined for failing to protect personal information (POPIA), and cannot work while its systems are down. **Safeguards** work best together: antivirus and a firewall keep malware out, encryption makes stolen data useless, strong passwords with two-step verification protect accounts, regular updates close security holes, regular backups allow recovery from ransomware, and training teaches staff to spot phishing, which is usually the way in.

6.7 Who am I? and what is the crime? [5]

Circle the best answer for each.

1	I break the copy protection on software so that it can be shared.	**HACKER** **CRACKER** **CYBER GANG**
2	A group of us share the work: some write the malware, some steal the data and some sell it.	**HACKER** **CRACKER** **CYBER GANG**
3	A criminal uses a stolen ID number to take out a loan in another person's name.	**PHISHING** **ID THEFT** **RANSOMWARE** **BOTNET**
4	All the files on a company's server are locked and a message demands payment.	**PHISHING** **ID THEFT** **RANSOMWARE** **BOTNET**
5	Thousands of hijacked home computers send traffic to take a website offline.	**PHISHING** **ID THEFT** **RANSOMWARE** **BOTNET**

6.8 Write it: the scenario **[8]**

On Monday morning the staff of a small accounting firm find that every file on the office server is locked. A message demands R30 000. The attack began when an employee opened an email that looked like it came from a client. Write a **paragraph** that (a) names the type of cybercrime and the way it got in, (b) explains **two effects** on the firm and (c) recommends **two safeguards**, saying how each helps.

TERM CHECKPOINT

Mid-year checkpoint: objects, grids and the connected world

Time: 120 minutes	**Total:** 79 marks	**Mark:** _____ / 79

No notes, no computer. Show all your working. Write Delphi code neatly and use correct indentation.

SECTION A — Object-oriented programming [31]

*Use the class diagram of **TTaxiTrip** in A1 for the whole section. Add **uses SysUtils** where needed; you may leave out the unit and implementation headings.*

A1 [5]

A taxi association wants to record every trip. Here is the class diagram for **TTaxiTrip**. A taxi seats **15** passengers: use a constant **MAXSEATS**.

TTaxiTrip
- fRoute : string - fPassengers : Integer - fFare : Real (rand per passenger) - TotalIncome : Real (auxiliary)
+ Create + Create(sRoute : string; rFare : Real) + GetRoute : string + GetPassengers : Integer + SetFare(rNew : Real) + AddPassengers(iNum : Integer) : Boolean + IsFull : Boolean + ToString : string

Write the **constant and the type declaration** of the class (the interface part only), exactly as in the diagram. The auxiliary method is private; the rest are public.

A2 **[4]**

Write the implementation of **both constructors**. The parameterised one stores the route and fare and starts with 0 passengers. The non-parameterised one makes a trip called **Not set** with 0 passengers and a fare of 0.

A3 **[4]**

Write the accessor **GetPassengers** and the mutator **SetFare**. SetFare must only change the fare if the new fare is greater than 0.

A4 **[5]**

Write the method **AddPassengers(iNum)**. It adds iNum passengers only if iNum is more than 0 **and** the total number of passengers would not be more than MAXSEATS. It returns True if the passengers were added and False if not.

A5 **[6]**

Write the private auxiliary function **TotalIncome** (passengers × fare), the public Boolean function **IsFull** (True when the taxi has exactly MAXSEATS passengers) and **ToString**, which returns a sentence such as *Bellville - Parow: 15 passengers, income R210,00*. ToString must call TotalIncome and show the income with two decimals.

A6 **[7]**

The form has **edtRoute**, **edtFare**, **edtNum**, **lblTrip**, **lblMsg** and **btnLoad**. Write the code for btnLoad. It must create a trip with the route and the fare, add the number of passengers in edtNum, show the trip's ToString in lblTrip if they fit, otherwise show **Not enough seats** in lblMsg, and finally show **Ready to leave** in lblMsg if the taxi is full.

SECTION B — Two-dimensional arrays [17]

B1 [5]

The taxi rank records how many passengers were loaded on each of 4 routes (rows) in the morning, at midday and in the afternoon (columns 1 to 3). Sample data in the array **arrLoad**:

arrLoad	1 Morning	2 Midday	3 Afternoon
1 Route 1	38	52	41
2 Route 2	25	47	30
3 Route 3	60	64	58
4 Route 4	19	33	27

Write the **constants** ROUTES and TIMES, the **declaration** of arrLoad (whole numbers) and of the loop variables r and c, and nested loops that fill every cell with a number typed by the user in an InputBox.

B2 [5]

Write code that displays the **total for every route** in **memOut** in the form *Route 2: 102*. (With the sample data the four lines are *Route 1: 131*, *Route 2: 102*, *Route 3: 182*, *Route 4: 79*.)

B3 **[3]**

Write code that calculates the total number of passengers loaded in the **afternoon** (column 3) and shows it in **lblAfternoon**. (Sample data: 156.)

B4 **[4]**

Write code that finds the **number of the route with the highest total** (all three times added) and shows it in **lblBest**. (Sample data: route 3, total 182.)

SECTION C — Trace and find the errors [14]

C1 [6]

Trace the calls on **objTrip**. The rules are those of **TTaxiTrip** in section A (the taxi seats 15; a refused call changes nothing). Write the passengers and fare **after** each call, and anything a call returns.

```Delphi
objTrip := TTaxiTrip.Create('Bellville - Parow', 12);
objTrip.AddPassengers(9);
objTrip.AddPassengers(4);
objTrip.AddPassengers(3);
objTrip.IsFull;
objTrip.AddPassengers(2);
objTrip.IsFull;
objTrip.SetFare(0);
objTrip.SetFare(14);
objTrip.ToString;
```

Call	fPassengers	fFare	Returns
objTrip := TTaxiTrip.Create('Bellville - Parow', 12)			
objTrip.AddPassengers(9)			
objTrip.AddPassengers(4)			
objTrip.AddPassengers(3)			
objTrip.IsFull			
objTrip.AddPassengers(2)			
objTrip.IsFull			
objTrip.SetFare(0)			
objTrip.SetFare(14)			
objTrip.ToString			

C2 **[4]**

arrData holds the grid **5 2 9** / **4 8 1** / **7 3 6** (three rows). Trace the loop. Note that the inner loop stops at **r**.

Delphi

```
iTotal := 0;
for r := 1 to 3 do
  for c := 1 to r do
    iTotal := iTotal + arrData[r, c];
```

r	c	arrData[r, c]	iTotal
1	1		
2	1		
2	2		
3	1		
3	2		
3	3		

C3 **[4]**

This button click should create a trip, raise the fare to R14 and show the trip and its income. Find **four** errors and write the corrected lines. (*TotalIncome* and the attributes are private.)

Delphi

```
objTrip := TTaxiTrip.Create;
objTrip.SetFare := 14;
objTrip.AddPassengers(9);
objTrip.fPassengers := objTrip.fPassengers + 2;
memOut.Lines.Add(objTrip);
lblIncome.Caption := objTrip.TotalIncome;
```

SECTION D — Theory [17]

D1 [4]

Circle the best answer.

1	A learner receives an SMS that looks as if it is from her bank and asks her to confirm her PIN on a web page.	**PHISHING** **ID THEFT** **RANSOMWARE** **BOTNET**
2	A criminal takes out a store account in a stranger's name using the stranger's ID number.	**PHISHING** **ID THEFT** **RANSOMWARE** **BOTNET**
3	A clinic's patient files are locked until a payment is made.	**PHISHING** **ID THEFT** **RANSOMWARE** **BOTNET**
4	A network of hijacked computers is secretly used to flood a website with traffic.	**PHISHING** **ID THEFT** **RANSOMWARE** **BOTNET**

D2 [4]

A school replaces its old file server with a cloud storage service. **Discuss** two advantages of this move and its effect on the school's hardware. Write in sentences.

D3 [4]

A hospital uses AI to select patients for surgery. **Discuss** one advantage, one limitation and who should be accountable if the AI makes a wrong decision. Write in sentences.

D4 **[5]**

A cyber gang sends emails to a firm's staff. One employee opens an attachment and the firm's computers are infected with malware that steals customer information. Write a **paragraph** that names the crime, explains **one effect** on the firm and recommends **two safeguards**, saying how each helps.

COMING UP NEXT TERM · warm-up

Asking a smaller version of yourself

Next term you will meet a very odd idea in programming: a job that is solved by **asking a smaller copy of the same job**. No code is needed to understand it.

The queue. Five learners stand in a line facing forward. Nobody can see past the person in front. You are at the back and must find out **how many people are in the line, including you**, by passing one question forward. The rule is the same for everyone: *"Ask the person in front how many people are in the line from them forwards, then add 1 for yourself. If nobody is in front of you, say 1."* Number the learners from the **front (1)** to the **back (5)** and write the answer each one calls back:

Person	Asks the person in front?	The answer they call back
1 (front)		
2		
3		
4		
5 (back)		

Russian dolls. A set of nested dolls has an unknown number of dolls. You may open only one doll at a time. How could you count them using the same rule over and over? Write your rule in a sentence, and say what you do when you open a doll and find it is empty.